PROJECT 1

Student Management System

A beginner-friendly Java console application

Prepared by: Vallam Mounisha
Course: B.Tech – ECM
Repository name: Student-Management-System

Project Documentation

1. Introduction

The Student Management System is a simple Java console application designed to store and manage student details. It demonstrates basic programming concepts such as classes, objects, arrays, loops, conditional statements, and user input.

2. Objective

The objective is to make common student-record tasks easy to perform from a menu-driven program, while practicing Java fundamentals.

3. Technologies Used

• Programming language: Java
• Input: Scanner class
• Interface: Console/menu-driven

4. Main Features

• Add a new student record

• View all student records

• Search for a student by ID

• Update a student's details

• Delete a student record

• Exit the application

5. Java Source Code

```
import java.util.*;

class Student {
    int id;
    String name;
    int age;
    String course;

    Student(int id, String name, int age, String course) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.course = course;
    }

    void display() {
        System.out.println("ID: " + id + ", Name: " + name
                + ", Age: " + age + ", Course: " + course);
    }
}

public class Main {
    static Scanner sc = new Scanner(System.in);
    static Student[] students = new Student[100];
    static int count = 0;

    static int findStudent(int id) {
        for (int i = 0; i < count; i++) {
            if (students[i].id == id) return i;
        }
        return -1;
    }

    public static void main(String[] args) {
        int choice;
        do {
```

```
System.out.println("\\n--- Student Management System ---");
System.out.println("1. Add Student");
System.out.println("2. View Students");
System.out.println("3. Search Student");
System.out.println("4. Update Student");
System.out.println("5. Delete Student");
System.out.println("6. Exit");
System.out.print("Enter choice: ");
choice = sc.nextInt();
sc.nextLine();

switch (choice) {
    case 1:
        if (count == students.length) {
            System.out.println("Student list is full.");
            break;
        }
        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        if (findStudent(id) != -1) {
            System.out.println("Student ID already exists.");
            break;
        }
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter course: ");
        String course = sc.nextLine();
        students[count++] = new Student(id, name, age, course);
        System.out.println("Student added successfully.");
        break;

    case 2:
        if (count == 0) {
            System.out.println("No student records found.");
        } else {
            for (int i = 0; i < count; i++) {
                students[i].display();
            }
        }
        break;

    case 3:
        System.out.print("Enter student ID to search: ");
        id = sc.nextInt();
        int index = findStudent(id);
        if (index == -1) System.out.println("Student not found.");
        else students[index].display();
        break;

    case 4:
        System.out.print("Enter student ID to update: ");
        id = sc.nextInt();
        sc.nextLine();
        index = findStudent(id);
        if (index == -1) {
            System.out.println("Student not found.");
        } else {
            System.out.print("Enter new name: ");
            students[index].name = sc.nextLine();
            System.out.print("Enter new age: ");
            students[index].age = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter new course: ");
            students[index].course = sc.nextLine();
            System.out.println("Student updated successfully.");
        }
        break;

    case 5:
        System.out.print("Enter student ID to delete: ");
        id = sc.nextInt();
        index = findStudent(id);
        if (index == -1) {
            System.out.println("Student not found.");
        } else {
            for (int i = index; i < count - 1; i++) {
                students[i] = students[i + 1];
```

```
                        }
                        students[--count] = null;
                        System.out.println("Student deleted successfully.");
                    }
                    break;

                case 6:
                    System.out.println("Thank you for using the system.");
                    break;

                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 6);
        sc.close();
    }
}
```

6. Sample Run

```
--- Student Management System ---
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter choice: 1
Enter ID: 101
Enter name: Mounisha
Enter age: 18
Enter course: B.Tech ECM
Student added successfully.

Enter choice: 2
ID: 101, Name: Mounisha, Age: 18, Course: B.Tech ECM

Enter choice: 6
Thank you for using the system.
```

7. Expected Output

When a user adds student ID 101 with the name Mounisha, age 18, and course B.Tech ECM, the program confirms that the record was added. Choosing View Students displays the stored record. The menu continues until Exit is selected.

8. How to Run

1. Save the source code in a file named **Main.java**.
2. Open a terminal in that folder.
3. Compile using **javac Main.java**.
4. Run using **java Main**.

9. Conclusion

This project provides practice with Java classes, objects, arrays, loops, methods, and menu-based input. It can be extended in the future by storing records in a file or database and adding stronger input validation.

GitHub repository: Student-Management-System